

Chapitre 3

Implémentation et Résultats Expérimentaux

Ce chapitre présente l'implémentation détaillée de l'algorithme de segmentation et compression asymétrique proposé pour les réseaux de capteurs multimédias en agriculture de précision. Nous décrivons d'abord l'architecture globale du système (section 3.1), puis les implémentations spécifiques du traitement côté nœud capteur (section 3.2) et côté puits (section 3.3). Les outils d'analyse développés sont présentés dans la section 3.4, suivis du protocole expérimental (section 3.5) et des résultats obtenus (section 3.6). Une analyse critique et comparative avec les méthodes standards (JPEG, JPEG 2000 ROI) est proposée dans la section 3.7.

3.1 Architecture générale du système

3.1.1 Vue d'ensemble de l'implémentation

L'architecture du système proposé repose sur un paradigme de traitement asymétrique distribué entre le nœud capteur et la station de base (puits), conformément aux principes établis par Akyildiz et al. [1] pour les réseaux de capteurs sans fil multimédias. Cette séparation fonctionnelle permet d'exploiter l'hétérogénéité des ressources disponibles : le nœud capteur, fortement contraint en énergie, calcul et mémoire, effectue uniquement les opérations critiques minimales, tandis que le puits, disposant de ressources abondantes, prend en charge les traitements complexes de reconstruction et d'amélioration.

Le traitement côté nœud capteur a été implémenté en C++ avec la bibliothèque OpenCV 4.x [?], offrant un compromis optimal entre performance et portabilité vers des plateformes embarquées. Ce choix se justifie par la maturité de l'écosystème C++ pour les systèmes contraints et la disponibilité d'optimisations bas niveau nécessaires à la réduction de la consommation énergétique. Le traitement côté puits a été développé en Python 3.10+ en utilisant les bibliothèques scikit-image [?] et OpenCV, privilégiant ainsi la flexibilité et la rapidité de développement pour l'expérimentation d'algorithmes de reconstruction avancés.

Cette architecture modulaire permet une validation séparée de chaque composant et facilite l'évaluation du compromis énergie-qualité à travers des métriques distinctes pour le nœud (consommation énergétique, taux de compression) et le puits (qualité de reconstruction, temps de traitement).

CHAPITRE 3. IMPLÉMENTATION ET RÉSULTATS EXPÉRIMENTAUX

3.1.2 Diagramme de flux de données

La figure 3.1 illustre le flux de données complet du système, depuis l'acquisition de l'image brute jusqu'à la reconstruction finale. Le pipeline se divise en trois phases principales :

Phase 1 - Traitement nœud capteur : l'image acquise subit une série de transformations légères visant à réduire le volume de données tout en préservant l'information pertinente. Cette phase comprend l'extraction d'indicateurs visuels (indices de végétation, histogrammes), la détection de changement par rapport à une référence temporelle, la segmentation orientée anomalies et la dégradation contrôlée différenciée selon les régions d'intérêt.

Phase 2 - Transmission : les données transmises se limitent à l'image dégradée, au masque binaire de segmentation et aux indicateurs visuels compacts. Ce paquet minimal exploite le principe de compression asymétrique où le coût énergétique de la transmission radio, dominant dans les réseaux de capteurs sans fil [?], est drastiquement réduit.

Phase 3 - Reconstruction au puits : le puits décode les données reçues et applique un pipeline d'amélioration progressive combinant upsampling intelligent, débruitage adaptatif, filtrage préservant les contours et rehaussement localisé des régions d'intérêt. Cette approche multi-étapes s'inspire des travaux sur la super-résolution et l'amélioration d'images guidée par segmentation [?].

FIGURE 3.1 – Architecture globale du système : traitement asymétrique nœud-puits. Le nœud effectue une compression sémantique légère tandis que le puits reconstruit l'image avec des algorithmes avancés.

3.1.3 Technologies et outils utilisés

La tableau 3.1 récapitule l'ensemble des technologies et bibliothèques utilisées pour l'implémentation du système. Le choix d'OpenCV s'impose comme standard de facto pour le traitement d'images en temps réel, offrant des primitives optimisées compatibles avec les contraintes embarquées. La bibliothèque scikit-image [?] fournit des implémentations de référence pour les métriques de qualité d'image (SSIM, PSNR) largement utilisées dans la littérature. Les outils d'analyse et de visualisation (matplotlib, numpy) permettent une évaluation reproductible et une comparaison systématique avec les méthodes de l'état de l'art.

TABLE 3.1 – Technologies et bibliothèques utilisées dans l'implémentation

Composant	Technologie	Version
Traitement nœud	C++ / OpenCV	4.x
Traitement puits	Python	3.10+
Métriques qualité	scikit-image	0.21+
Analyse	matplotlib, numpy	-
Compilation	GCC	11.x
Gestion mémoire	rusage (POSIX)	-

3.1.4 Organisation modulaire du code

L'implémentation adopte une architecture modulaire en trois composants principaux, favorisant la séparation des préoccupations et la testabilité indépendante de chaque module :

Module `node_process.c` : ce module implémente l'intégralité du traitement côté nœud capteur, depuis l'acquisition de l'image jusqu'à la préparation des données pour transmission. Il encapsule les structures de données (profils énergétiques, indicateurs visuels, métriques), les algorithmes de segmentation et de dégradation, ainsi que le modèle d'estimation énergétique. La compilation produit un exécutable autonome acceptant comme paramètres l'image d'entrée, le répertoire de sortie et le profil énergétique souhaité (E ou Q).

Module `sink_reconstruct.py` : ce module Python implémente le pipeline de reconstruction au puits sous forme d'une classe `SinkReconstructor`. Il charge les données transmises (image dégradée, masque, indicateurs), applique les algorithmes de reconstruction et d'amélioration progressive, puis calcule les métriques de qualité par comparaison avec l'image originale. L'approche orientée objet facilite l'extension du pipeline avec de nouveaux algorithmes d'amélioration.

Module `analyze_results.py` : ce module d'analyse automatisée permet l'évaluation comparative de plusieurs configurations ou méthodes. Il charge les métriques sauvegardées par les modules précédents, calcule un score d'efficacité global combinant énergie et qualité, génère des visualisations comparatives (graphiques de compromis, tableaux récapitulatifs) et exporte les résultats au format JSON pour faciliter l'intégration dans des workflows d'expérimentation plus larges. Cette approche suit les recommandations de reproductibilité de la recherche en traitement d'images [?].

3.2 Implémentation du traitement côté nœud capteur

3.2.1 Gestion des profils énergétiques

Structure des profils E et Q

La gestion de l'énergie repose sur deux profils paramétriques prédéfinis qui permettent d'adapter le compromis qualité-énergie sans modification algorithmique. Cette approche s'inspire des travaux sur la qualité de service adaptative dans les réseaux de capteurs multi-médias [?], où l'ajustement dynamique des paramètres permet de répondre aux contraintes énergétiques tout en maintenant une qualité minimale acceptable pour l'application.

Le Listing 3.1 présente la structure de données utilisée pour encapsuler les paramètres de chaque profil. Cette représentation compacte regroupe l'ensemble des seuils et configurations influençant le pipeline de traitement, depuis la résolution spatiale jusqu'aux niveaux de quantification différenciés entre régions d'intérêt et fond de l'image.

```
1 typedef struct {
2     char name[10];
3     int resolution_factor;           // Sous-echantillonnage (1, 2, 4)
4     int block_size;                 // Taille blocs (8, 16, 32)
5     int quantization_levels;        // Niveaux quantification
6     float anomaly_threshold;        // Seuil detection anomalie
7     int roi_quality;                // Qualite ROI (1-100)
8     int bg_quality;                 // Qualite fond (1-100)
```

CHAPITRE 3. IMPLÉMENTATION ET RÉSULTATS EXPÉRIMENTAUX

```
9 } EnergyProfile;  
10  
11 // Profils pre-configures  
12 EnergyProfile PROFILE_E = {"E", 4, 16, 16, 0.25, 40, 10};  
13 EnergyProfile PROFILE_Q = {"Q", 2, 8, 64, 0.15, 75, 30};
```

Listing 3.1 – Structure des profils énergétiques

Le **profil E (Energy-first)** privilégie la longévité du nœud capteur en maximisant les économies énergétiques. Il utilise un sous-échantillonnage spatial agressif (facteur 4), des blocs de grande taille pour la segmentation (16×16 pixels) réduisant le nombre d'opérations, et une quantification grossière (16 niveaux). Le seuil de détection d'anomalie est fixé à 0.25, favorisant une segmentation conservatrice qui limite la taille des régions d'intérêt et donc le volume de données à transmettre.

Le **profil Q (Quality-first)** optimise la qualité de reconstruction tout en maintenant un contrôle sur les coûts énergétiques. Le sous-échantillonnage modéré (facteur 2) et les blocs fins (8×8 pixels) permettent une segmentation plus précise. La quantification à 64 niveaux préserve davantage de détails, tandis que le seuil d'anomalie abaissé (0.15) détecte plus finement les zones d'intérêt. Ce profil correspond aux situations où l'énergie est disponible (batterie chargée, ensoleillement suffisant) et où la précision de l'observation est critique.

Paramètres de configuration

La tableau 3.2 présente une comparaison détaillée des paramètres des deux profils énergétiques. Les valeurs ont été déterminées empiriquement à travers une série d'expérimentations préliminaires visant à identifier les seuils optimaux pour chaque configuration. Le facteur de résolution, paramètre le plus influent sur la consommation énergétique, varie d'un facteur 2 entre les deux profils, ce qui correspond à une réduction de 75% du volume de données pour le profil E par rapport à l'image originale (facteur 4), contre 75% pour le profil Q (facteur 2).

La taille de bloc influence directement la granularité de la segmentation et le nombre d'opérations de comparaison nécessaires. Des blocs de 16×16 pixels (profil E) réduisent la complexité computationnelle d'un facteur 4 par rapport aux blocs de 8×8 (profil Q), au prix d'une précision de segmentation moindre. Cette approche s'aligne sur les observations de Wang et al. [?] sur l'importance du choix de la taille de bloc dans les algorithmes de compression orientés contenu.

Les niveaux de quantification différenciés entre ROI et fond constituent une innovation clé de notre approche. En allouant proportionnellement plus de bits aux régions d'intérêt (40% vs 10% en mode E, 75% vs 30% en mode Q), le système préserve l'information pertinente tout en compressant agressivement les zones de faible intérêt, conformément aux principes de la compression basée régions [16].

3.2.2 Extraction des indicateurs visuels

Calcul des indices de végétation

Les indicateurs visuels sont calculés en une seule passe pour minimiser la complexité computationnelle, respectant ainsi la contrainte de frugalité énergétique du nœud capteur. L'approche privilégie des indices simples, calculables sans transformations complexes, tout

CHAPITRE 3. IMPLÉMENTATION ET RÉSULTATS EXPÉRIMENTAUX

TABLE 3.2 – Paramètres des profils énergétiques E et Q

Paramètre	Profil E	Profil Q
Facteur de résolution	4	2
Taille de bloc (pixels)	16	8
Niveaux de quantification	16	64
Seuil d'anomalie	0.25	0.15
Qualité ROI (%)	40	75
Qualité fond (%)	10	30

en préservant un pouvoir discriminant suffisant pour la détection de changements et la caractérisation de la scène.

Deux indices de végétation largement utilisés en télédétection agricole sont implémentés. L'**Excess Green Index** (ExG), proposé par Woebbecke et al. [?], met en évidence les pixels à dominante verte, caractéristiques de la végétation active :

$$\text{ExG} = 2g - r - b \quad (3.1)$$

Le **Normalized Green-Red Difference Index** (NGRDI), introduit par Tucker [?], normalise la différence entre les canaux vert et rouge, offrant une meilleure robustesse aux variations d'illumination :

$$\text{NGRDI} = \frac{g - r}{g + r + \epsilon} \quad (3.2)$$

où r , g , b représentent les composantes normalisées RGB dans l'intervalle $[0, 1]$, et $\epsilon = 0.001$ prévient la division par zéro. Ces indices permettent une détection précoce du stress végétal, des variations de croissance et de l'apparition de fruits, aspects cruciaux en agriculture de précision [?].

En complément, des histogrammes grossiers à 8 bins par canal sont calculés simultanément, fournissant une signature colorimétrique compacte de la scène. Cette représentation réduite (24 valeurs entières) capture les caractéristiques chromatiques globales tout en maintenant un coût mémoire et computationnel minimal.

Histogrammes et statistiques globales

Le Listing 3.2 présente l'implémentation de l'extraction des indicateurs visuels. L'algorithme effectue une unique traversée de l'image, accumulant simultanément les statistiques colorimétriques, les indices de végétation et les histogrammes. Cette stratégie mono-passe minimise les accès mémoire, facteur critique de consommation énergétique dans les systèmes embarqués [?].

```

1 VisualIndicators extract_visual_indicators(Mat& img) {
2     VisualIndicators ind = {0};
3     int total_pixels = img.rows * img.cols;
4     double sum_r = 0, sum_g = 0, sum_b = 0;
5     double sum_exg = 0, sum_ngrdi = 0;
6
7     // Une seule passe sur l'image
8     for(int y = 0; y < img.rows; y++) {
9         for(int x = 0; x < img.cols; x++) {
```



CHAPITRE 3. IMPLÉMENTATION ET RÉSULTATS EXPÉRIMENTAUX

```
10      Vec3b pixel = img.at<Vec3b>(y, x);
11      float b = pixel[0] / 255.0;
12      float g = pixel[1] / 255.0;
13      float r = pixel[2] / 255.0;
14
15      sum_r += r; sum_g += g; sum_b += b;
16
17      // Indices de vegetation
18      float exg = 2*g - r - b;
19      float ngrdi = (g - r) / (g + r + 0.001);
20      sum_exg += exg;
21      sum_ngrdi += ngrdi;
22
23      // Histogrammes grossiers (8 bins)
24      ind.hist_r[(int)(r * 7.999)]++;
25      ind.hist_g[(int)(g * 7.999)]++;
26      ind.hist_b[(int)(b * 7.999)]++;
27  }
28 }
29
30 ind.exg_mean = sum_exg / total_pixels;
31 ind.ngrdi_mean = sum_ngrdi / total_pixels;
32 return ind;
33 }
```

Listing 3.2 – Extraction des indicateurs visuels en une passe

L'utilisation d'histogrammes à 8 bins (au lieu des 256 bins traditionnels) réduit drastiquement l'empreinte mémoire tout en conservant une résolution suffisante pour caractériser la distribution colorimétrique. Cette granularité s'avère adéquate pour des scènes agricoles typiquement dominées par quelques couleurs principales (vert de la végétation, brun du sol, couleurs vives des fruits). La variance globale, calculée via les primitives OpenCV optimisées, complète cette signature en quantifiant la complexité texturale de la scène.

Complexité algorithmique

La complexité temporelle de l'extraction est strictement linéaire : $O(N)$ où $N = W \times H$ représente le nombre de pixels de l'image. Chaque pixel est visité exactement une fois, subissant un nombre constant d'opérations élémentaires (additions, divisions, indexation). Cette propriété garantit un temps d'exécution prédictible et proportionnel à la résolution de l'image, aspect crucial pour les systèmes temps réel contraints.

La complexité spatiale est également optimale : $O(1)$ pour les accumulateurs scalaires (moyennes, sommes) et $O(24)$ pour les histogrammes tricolores à 8 bins, soit une empreinte mémoire constante indépendante de la taille de l'image. Cette caractéristique permet l'implémentation sur des microcontrôleurs à mémoire limitée, typiquement quelques dizaines de kilooctets de RAM disponible [?].

En pratique, pour une image de résolution modeste (640×480 pixels, typique des capteurs embarqués économes en énergie), l'algorithme nécessite environ 307 200 itérations de la boucle principale, chacune effectuant 15 opérations élémentaires (normalisations, calculs d'indices, incrémentations), soit un total approximatif de 4,6 millions d'opéra-



CHAPITRE 3. IMPLÉMENTATION ET RÉSULTATS EXPÉRIMENTAUX

tions. Sur un processeur ARM Cortex-M4 cadencé à 168 MHz (plateforme typique pour WSN multimédias), cela correspond à un temps d'exécution théorique de l'ordre de 30 ms, confirmant la viabilité de l'approche pour des applications temps réel.

3.2.3 Détection de changement

Comparaison avec référence temporelle

Le mécanisme de détection de changement exploite la redondance temporelle inhérente aux scènes agricoles, caractérisées par une dynamique lente à l'échelle horaire ou journalière [?]. En comparant les indicateurs visuels de l'image courante avec ceux d'une référence temporelle (dernière image transmise), le système peut décider de supprimer la transmission si aucun changement significatif n'est détecté, réalisant ainsi des économies énergétiques substantielles.

La métrique de distance utilisée combine linéairement plusieurs composantes, pondérées selon leur pertinence pour la détection de changements pertinents en contexte agricole :

$$d(\mathbf{I}_t, \mathbf{I}_{\text{ref}}) = \sum_{c \in \{R, G, B\}} |\mu_c^t - \mu_c^{\text{ref}}| + 0.5 \cdot |\text{ExG}_t - \text{ExG}_{\text{ref}}| + 0.5 \cdot |\text{NGRDI}_t - \text{NGRDI}_{\text{ref}}| + \frac{|\sigma_t - \sigma_{\text{ref}}|}{100} \quad (3.3)$$

où μ_c représente la moyenne du canal c , ExG et NGRDI les indices de végétation, et σ la variance globale. Les coefficients de pondération (0.5 pour les indices, 1/100 pour la variance) ont été ajustés empiriquement pour équilibrer les contributions des différentes composantes.

Cette approche présente plusieurs avantages. Premièrement, elle évite le calcul coûteux de métriques pixel à pixel (corrélation, différence absolue moyenne), se contentant de comparer des scalaires. Deuxièmement, elle est robuste aux variations d'illumination mineures grâce à la normalisation des indices de végétation. Troisièmement, elle permet un ajustement fin de la sensibilité via le seuil θ , paramétrable selon l'application et les contraintes énergétiques.

Mécanisme de décision

Le Listing 3.3 présente l'implémentation de la fonction de détection de changement. La décision binaire (transmission ou suppression) repose sur la comparaison de la distance calculée avec un seuil θ fixe, typiquement compris entre 0.05 et 0.2 selon l'application.

```
1 int detect_change(VisualIndicators& current ,
2                   VisualIndicators& reference ,
3                   float threshold) {
4     float diff = 0;
5     diff += fabs(current.mean_r - reference.mean_r);
6     diff += fabs(current.mean_g - reference.mean_g);
7     diff += fabs(current.mean_b - reference.mean_b);
8     diff += fabs(current.exg_mean - reference.exg_mean) * 0.5;
9     diff += fabs(current.ngrdi_mean - reference.ngrdi_mean) * 0.5;
10
11     return (diff > threshold) ? 1 : 0;
12 }
```





Listing 3.3 – Détection de changement par comparaison d’indicateurs

En cas d’absence de changement détecté, le nœud transmet un paquet minimal (quelques octets) indiquant la stabilité de la scène, évitant ainsi la transmission de l’image complète. Cette stratégie s’apparente aux techniques de compression vidéo inter-trame [18], adaptées ici au contexte de surveillance agricole à faible fréquence d’acquisition (une image toutes les 10-30 minutes typiquement).

L’économie énergétique potentielle est considérable : pour une scène stable sur une période de 2 heures avec acquisition toutes les 15 minutes, seule la première image est transmise intégralement, les 7 acquisitions suivantes générant uniquement des paquets de notification de quelques dizaines d’octets, réduisant le coût de transmission d’un facteur 100 ou plus. Cette approche rejoint les principes d’échantillonnage adaptatif et de transmission événementielle proposés pour les réseaux de capteurs économes en énergie [?].

3.2.4 Segmentation légère orientée anomalies

Approche par blocs

La segmentation repose sur une analyse par blocs de taille fixe définie par le profil énergétique, évitant ainsi les algorithmes de segmentation sémantique complexes (deep learning, graph cuts) inadaptés aux contraintes du nœud capteur. Cette approche délibérément simple s’inspire des méthodes de segmentation par régions homogènes [?], adaptées ici pour une exécution déterministe à coût prédictible.

Le choix d’une grille régulière présente plusieurs avantages. Premièrement, l’indexation des blocs est triviale, évitant les structures de données complexes. Deuxièmement, le nombre d’opérations est connu a priori : pour une image $W \times H$ et des blocs de taille $b \times b$, exactement $\lceil W/b \rceil \times \lceil H/b \rceil$ blocs sont analysés. Troisièmement, cette approche est naturellement parallélisable, chaque bloc étant traité indépendamment.

La figure 3.2 illustre le processus complet : l’image originale est divisée en blocs réguliers, chaque bloc est évalué selon sa déviation par rapport au comportement moyen de la scène, et un masque binaire est généré marquant les blocs significativement différents comme régions d’intérêt. Cette segmentation non sémantique capture naturellement les anomalies visuelles (stress végétal, fruits, maladies) sans nécessiter de modèle préalable de la scène.

FIGURE 3.2 – Processus de segmentation par blocs : (a) image originale de culture maraîchère, (b) grille de blocs 16×16 superposée, (c) masque binaire de segmentation identifiant les régions d’intérêt (fruits et zones de stress en blanc)

Détection de déviation locale

Pour chaque bloc B_i , la déviation par rapport à la moyenne globale μ_g de l’image est calculée comme une distance normalisée dans l’espace colorimétrique RGB :

$$\delta(B_i) = \frac{1}{3 \cdot 255} \sum_{c \in \{R, G, B\}} |\mu_{B_i}^c - \mu_g^c| \quad (3.4)$$



CHAPITRE 3. IMPLÉMENTATION ET RÉSULTATS EXPÉRIMENTAUX

où $\mu_{B_i}^c$ représente la valeur moyenne du canal c dans le bloc B_i , et μ_g^c la moyenne globale sur l'ensemble de l'image. La normalisation par $3 \cdot 255$ ramène la déviation dans l'intervalle $[0, 1]$, facilitant la définition de seuils intuitifs.

Un bloc est marqué comme région d'intérêt si $\delta(B_i) > \theta_a$, où θ_a est le seuil de détection d'anomalie défini par le profil énergétique (tableau 3.2). Ce critère simple mais efficace permet d'identifier automatiquement plusieurs catégories d'éléments d'intérêt en agriculture de précision :

- **Fruits mûrs** : leur couleur distinctive (rouge, jaune, orange) contraste fortement avec le fond vert du feuillage, générant des déviations élevées.
- **Stress végétal** : les zones de jaunissement ou de nécrose présentent des signatures colorimétriques anormales par rapport au vert sain dominant.
- **Maladies et ravageurs** : les altérations de texture et de couleur (taches, décolorations) sont détectées comme déviations locales.
- **Objets étrangers** : éléments non végétaux (outils, déchets) s'écartant significativement de la palette colorimétrique naturelle.

Cette approche orientée anomalies, inspirée des travaux de Chandola et al. [?] en détection d'anomalies, s'avère particulièrement adaptée aux applications de surveillance où les événements d'intérêt sont relativement rares et se distinguent du comportement nominal de la scène.

Génération du masque de segmentation

```
1  Mat segment_anomalies(Mat& img, EnergyProfile& profile) {
2      Mat mask = Mat::zeros(img.rows, img.cols, CV_8U);
3      int bs = profile.block_size;
4      Scalar global_mean = mean(img);
5
6      // Parcours par blocs
7      for(int y = 0; y < img.rows; y += bs) {
8          for(int x = 0; x < img.cols; x += bs) {
9              int h = std::min(bs, img.rows - y);
10             int w = std::min(bs, img.cols - x);
11
12             Rect block_rect(x, y, w, h);
13             Mat block = img(block_rect);
14             Scalar block_mean = mean(block);
15
16             // Deviation normalisee
17             float deviation = 0;
18             for(int c = 0; c < 3; c++) {
19                 deviation += fabs(block_mean[c] - global_mean[c]);
20             }
21             deviation /= (3.0 * 255.0);
22
23             // Marquer comme ROI si deviation significative
24             if(deviation > profile.anomaly_threshold) {
25                 rectangle(mask, block_rect, Scalar(255), FILLED);
26             }
27         }
28     }
```



```
29     return mask;  
30 }
```

Listing 3.4 – Segmentation orientée anomalies

FIGURE 3.3 – Exemples de segmentation : détection automatique de fruits et zones de stress végétal

3.2.5 Dégradation contrôlée

Sous-échantillonnage spatial

Quantification différenciée ROI/fond

```
1  Mat degrade_image(Mat& img, Mat& mask, EnergyProfile& profile) {  
2      Mat degraded;  
3  
4      // Sous-echantillonnage spatial  
5      if(profile.resolution_factor > 1) {  
6          resize(img, degraded, Size(),  
7                  1.0/profile.resolution_factor,  
8                  1.0/profile.resolution_factor, INTER_AREA);  
9          resize(mask, mask, degraded.size(), 0, 0, INTER_NEAREST);  
10     } else {  
11         degraded = img.clone();  
12     }  
13  
14     // Quantification différenciée  
15     for(int y = 0; y < degraded.rows; y++) {  
16         for(int x = 0; x < degraded.cols; x++) {  
17             Vec3b& pixel = degraded.at<Vec3b>(y, x);  
18             int is_roi = mask.at<uchar>(y, x) > 0;  
19  
20             // Plus de niveaux pour ROI  
21             int levels = is_roi ? profile.quantization_levels :  
22                                 profile.quantization_levels / 4;  
23  
24             for(int c = 0; c < 3; c++) {  
25                 int quantized = (pixel[c] * levels) / 256;  
26                 pixel[c] = (quantized * 256) / levels;  
27             }  
28         }  
29     }  
30     return degraded;  
31 }
```

Listing 3.5 – Dégradation contrôlée avec traitement différencié

3.2.6 Métriques de performance du nœud

Mesure du temps CPU

Consommation mémoire

Estimation énergétique

Le modèle énergétique simplifié repose sur trois composantes principales :

$$E_{\text{CPU}} = P_{\text{CPU}} \cdot t_{\text{proc}} \quad (3.5)$$

$$E_{\text{MEM}} = P_{\text{MEM}} \cdot S_{\text{img}} \quad (3.6)$$

$$E_{\text{TX}} = P_{\text{TX}} \cdot S_{\text{tx}} \cdot 8 \quad (3.7)$$

où $P_{\text{CPU}} = 10$ mW, $P_{\text{MEM}} = 1$ mW/KB, et $P_{\text{TX}} = 50$ mW pour la transmission radio.

```
1 double estimate_energy(double cpu_time_ms,
2                         int img_size_kb,
3                         int tx_size_kb) {
4     // Modele simplifie base sur la litterature WSN
5     double cpu_energy = (10.0 * cpu_time_ms) / 1000.0; // mJ
6     double mem_energy = (1.0 * img_size_kb) / 1000.0; // mJ
7     double tx_energy = (50.0 * tx_size_kb * 8) / 1000.0; // mJ
8
9     return cpu_energy + mem_energy + tx_energy;
10 }
```

Listing 3.6 – Estimation de l'énergie consommée

3.3 Implémentation du traitement côté puits

3.3.1 Reconstruction de base

Upsampling et interpolation

Restauration des dimensions originales

3.3.2 Pipeline d'amélioration progressive

FIGURE 3.4 – Pipeline de reconstruction au puits : amélioration progressive

Débruitage Non-Local Means

Filtrage bilatéral

Sharpening global et focalisé ROI

Amélioration du contraste (CLAHE)

3.3.3 Métriques de qualité

SSIM (Structural Similarity Index)

L'indice SSIM mesure la similarité structurelle entre deux images :

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)} \quad (3.8)$$

PSNR (Peak Signal-to-Noise Ratio)

$$\text{PSNR} = 10 \log_{10} \left(\frac{\text{MAX}^2}{\text{MSE}} \right) \quad (3.9)$$

MSE (Mean Squared Error)

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (x_i - y_i)^2 \quad (3.10)$$

3.4 Outils d'analyse et de comparaison

3.4.1 Module d'analyse automatisée

3.4.2 Calcul du score d'efficacité

Un score d'efficacité global est calculé comme combinaison pondérée de trois facteurs :

$$S_{\text{eff}} = 0.4 \cdot S_E + 0.3 \cdot S_C + 0.3 \cdot S_Q \quad (3.11)$$

où S_E est le score énergétique (inversé), S_C le score de compression, et S_Q le score de qualité basé sur SSIM.

3.4.3 Génération de visualisations comparatives

3.5 Protocole expérimental

3.5.1 Jeu de données

Caractéristiques des images de test

Scénarios agricoles couverts

3.5.2 Configuration des tests

Paramètres des profils E et Q

Conditions de comparaison

3.5.3 Méthodes de référence

JPEG standard

JPEG 2000 avec ROI

Paramètres de configuration

TABLE 3.3 – Configuration des méthodes de référence

Méthode	Paramètre	Valeur
JPEG	Qualité	40, 75
JPEG 2000 ROI	Qualité ROI	75
JPEG 2000 ROI	Qualité fond	30
Proposée (E)	Voir profil	Table 3.2
Proposée (Q)	Voir profil	Table 3.2

3.6 Résultats expérimentaux

3.6.1 Performance énergétique

Consommation CPU et temps de traitement

TABLE 3.4 – Métriques de performance du nœud capteur

Méthode	CPU (ms)	Mémoire (KB)	Énergie (mJ)
Proposée - Profil E	X.XX	XXX	X.XXX
Proposée - Profil Q	X.XX	XXX	X.XXX
JPEG (Q=40)	X.XX	XXX	X.XXX
JPEG (Q=75)	X.XX	XXX	X.XXX
JPEG 2000 ROI	X.XX	XXX	X.XXX

Estimation énergétique globale

Taux de compression obtenus

3.6.2 Qualité de reconstruction

Métriques SSIM, PSNR, MSE

TABLE 3.5 – Métriques de qualité de reconstruction

Méthode	SSIM	PSNR (dB)	MSE
Proposée - Profil E	0.XXXX	XX.XX	XX.XX
Proposée - Profil Q	0.XXXX	XX.XX	XX.XX
JPEG (Q=40)	0.XXXX	XX.XX	XX.XX
JPEG (Q=75)	0.XXXX	XX.XX	XX.XX
JPEG 2000 ROI	0.XXXX	XX.XX	XX.XX

Analyse visuelle comparative

FIGURE 3.5 – Comparaison visuelle : (a) Original, (b) Profil E, (c) Profil Q, (d) JPEG Q=40, (e) JPEG Q=75, (f) JPEG 2000 ROI

3.6.3 Compromis énergie-qualité

Graphiques de trade-off

FIGURE 3.6 – Analyse des compromis : (a) Énergie vs SSIM, (b) Énergie vs PSNR, (c) Compression vs SSIM, (d) Scores d'efficacité